



UNIVERSIDADE DE BRASÍLIA
FACULDADE DE CIÊNCIAS E TECNOLOGIA EM ENGENHARIA

**RELATÓRIO DO PROJETO FINAL:
BYTE & BREW**

Membros do Grupo:

Bruno Cauê Souza Lopes – 252025423

Danielle de Sousa Pinheiro – 252014205

Lucas Miranda Souza – 252033630

Samyra Galvão de Carvalho – 252014949

Thamires Ellen Souza Araujo – 252027473

Relatório apresentado ao curso de Engenharia de Software da Universidade de Brasília - FCTE, para a disciplina de Orientação a Objetos, sob orientação do professor André Lanna.

BRASÍLIA - DF
06/07/2026

Sumário

1	Diagrama de Classes	2
2	Associações	2
3	Heranças	2
4	Polimorfismos	3
5	Exceções	3

1 Diagrama de Classes

O diagrama de classes que elaboramos está representado na Figura 1.

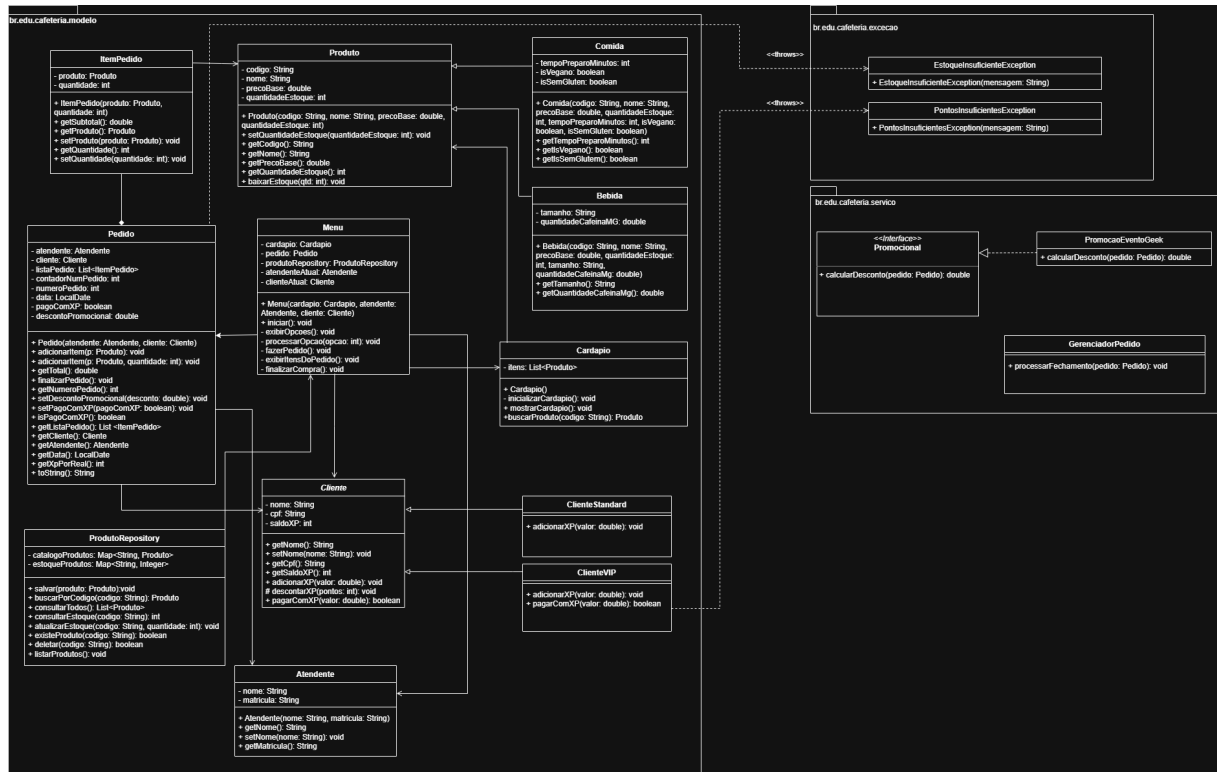


Figura 1: Diagrama de Classes

2 Associações

Para fazer a associação de classes, pensando no fluxo de uma venda, chegamos no seguinte:

- **Pedido, Cliente e Atendente:** qualquer instância de Pedido está diretamente associada a um objeto Atendente e a um objeto Cliente .
- **ItemPedido e Pedido:** uma instância de Pedido guarda uma lista inteira de itens, e cada um desses itens aponta para apenas uma instância de Produto.
- **Produto e ProdutoRepository:** a classe ProdutoRepository foi associada à classe Produto para cuidar do "banco de dados" do sistema, controlando o catálogo e o estoque.

3 Heranças

Para a aplicação de herança no projeto, focamos na organização do cardápio e dos clientes, dividindo as classes da seguinte forma:

- **Produto, Comida e Bebida:** A classe-mãe Produto concentra tudo o que é comum a qualquer item do cardápio (código, nome, preço base e quantidade em estoque). As classes filhas Comida e Bebida herdam essa estrutura básica e adicionam apenas os seus atributos específicos, como a quantidade de cafeína na Bebida ou o tempo de preparo e restrições alimentares na Comida.
- **Cliente, ClienteStandard e ClienteVIP:** A classe-mãe Cliente serve como uma base para guardar os dados principais de um cliente fidelidade (nome, CPF e saldoXP). As filhas ClienteStandard e ClienteVIP herdam essa base, mas especializam o comportamento, contendo os seus próprios métodos para a gestão dos pontos de experiência.

4 Polimorfismos

Para deixar o sistema mais flexível e evitar código repetido, usamos algumas estratégias de polimorfismo:

- **Sobrecarga:** Aplicamos o polimorfismo por sobrecarga na classe Pedido com o método adicionarItem. Criamos duas versões dele: uma mais rápida que recebe só o produto (assumindo que o cliente quer apenas 1 unidade) e outra mais completa que recebe o produto e a quantidade exata que o mesmo deseja.
- **Sobrescrita:** Aplicamos o polimorfismo por sobreescrita no método adicionarXP(valor: double). A classe-mãe Cliente tem a regra base, mas a classe ClienteVIP sobrescreve esse método para aplicar a regra do multiplicador de pontos dela na hora que a compra é feita.
- **Inclusão:** Aplicamos o polimorfismo por inclusão na integração entre Pedido, ItemPedido e as classes filhas (Comida e Bebida). A classe ItemPedido guarda apenas a classe-mãe genérica Produto. Como as classes Comida e Bebida herdam de Produto, o sistema aceita ambas naturalmente na mesma lista, sem precisar de validações extras ou de listas separadas para cada tipo.
- **Coerção:** Aplicamos o polimorfismo por coerção nas classes Cliente e ClienteVIP para a conversão de tipos de dados. Como o valor financeiro do pedido utiliza casas decimais (double), mas o saldo de pontos de experiência do cliente é um número inteiro (int), utilizamos o casting (int). Isso força a conversão de formato do dado, garantindo que o sistema transforme o dinheiro em pontos redondos sem causar erros.

5 Exceções

Para o tratamento de erros, utilizamos exceções customizadas da seguinte forma:

- **EstoqueInsuficienteException:** Criamos essa exceção para impedir o sistema de vender um produto que já acabou. Como ela herda da classe genérica Exception, ela obriga o sistema a prever e tratar esse erro. Se alguém tentar fechar um pedido de um item que não tem mais no estoque, o sistema lança o erro e barra a venda antes de computar qualquer pagamento.
- **PontosInsuficientesException:** Aplicada na regra de resgate de pontos do ClienteVIP. Como ela também herda da classe genérica Exception, ela também obriga o sistema a prever e tratar esse erro. Se o cliente tentar pagar a conta usando mais pontos do que realmente tem no saldo, essa exceção é lançada. Isso trava a finalização do pedido imediatamente, impedindo que o saldo do cliente fique negativo e garantindo a segurança da venda.
- **Tratamento de Erros (Try-Catch):** Implementamos a estrutura try-catch na classe Menu. Quando as classes lançam as exceções customizadas (como falta de estoque ou pontos), o bloco catch no menu intercepta esse erro antes que o programa seja interrompido. Em vez disso, o sistema exibe uma mensagem de alerta no console e mantém o programa rodando, retornando com segurança para as opções do menu principal.